Data 236- Distributed Systems

Homework-2

Kruthika Virupakshappa

Part 1 and 2

HTML, CSS and FastAPI

1. Write the code to add a new book. The user should be able to enter the Book Title and Author Name. Once the user submits the required data, the book should be added, and the user should be redirected to the home view showing the updated list of books.

   Explanation: Post API /api/books is used to add a new book. The backend receives the title and author from the frontend, removes extra spaces and checks that both fields are not empty. A new unique ID is created by finding the highest existing book ID and adding one. The new book is then added to the books list. Loadbooks() function is called to refresh the home view.

   Frontend: HTML

```
        <div class="actions-container">
            <h2>Add New Book</h2>
                <input type="text" id="addTitle" placeholder="Enter book title"/>
                <input type="text" id="addAuthor" placeholder="Enter author name"/>
                <button onclick="addBook()">Add Book</button>
        </div>
```

   Javascript:

```
async function addBook() {
    const titleEl = document.getElementById('addTitle');
    const authorEl = document.getElementById('addAuthor');
    const title = titleEl.value.trim();
    const author = authorEl.value.trim();
    if (!title || !author) {
        alert('Please provide both title and author.');
        return;
    }
    try {
        const res = await fetch(api_base, {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ title, author })
```

```
        });
        if (!res.ok) throw new Error('Failed to add book');
        titleEl.value = '';
        authorEl.value = '';
        await loadBooks();
    } catch (err) {
        console.error(err);
        alert('Could not add book. See console for details.');
    }
}
```

Backend (FastAPI): Post API is created to add a new book (api/books)

```
@app.post("/api/books", response_model=Book, status_code=201)
async def add_book(book_data: BookCreate):
    title = book_data.title.strip()
    author = book_data.author.strip()

    if not title:
        raise HTTPException(status_code=400, detail="Book title is required")
    if not author:
        raise HTTPException(status_code=400, detail="Author name is required")

    new_id = max([b.id for b in books], default=0) + 1
    new_book = Book(id=new_id, title=title, author=author)
    books.append(new_book)
    return new_book
```

User adds title and author, clicks on Add. Javascript sends the data to the backend. After it saves, we reload the homelist so the new book appears.

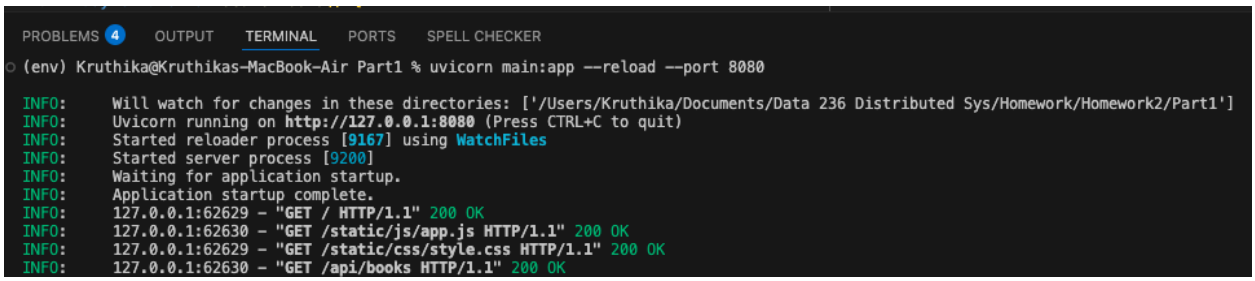
Style.css

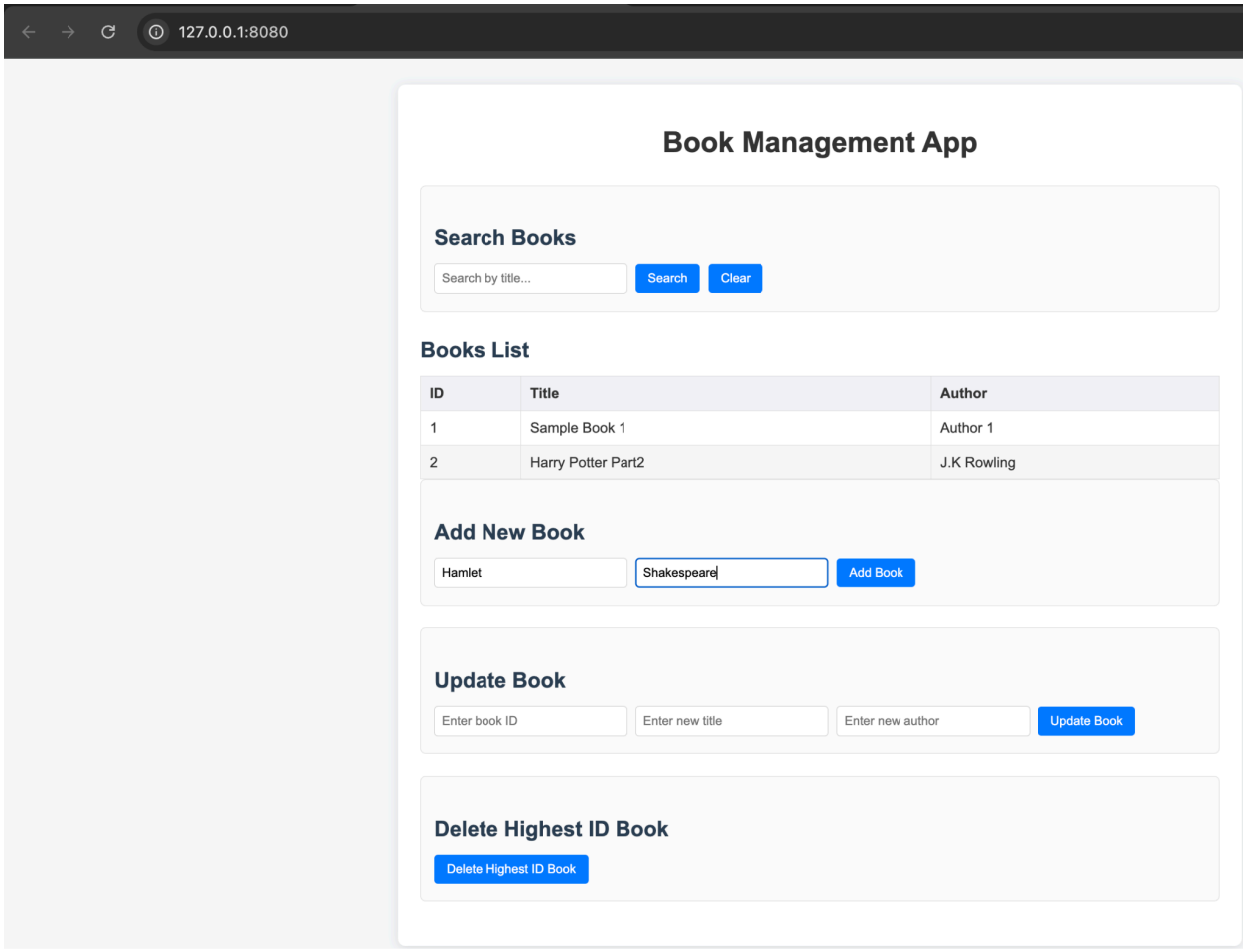
```
body {
    margin: 0;
    padding: 0;
    font-family: Arial, Helvetica, sans-serif;
    background-color: #f4f6f8;
    color: #333;
}

/* Main container */
.container {
    width: 80%;
    max-width: 900px;
    margin: 30px auto;
    background-color: #ffffff;
    padding: 25px;
```

```
    border-radius: 8px;
    box-shadow: 0 2px 10px rgba(0, 0, 0, 0.1);
}

/* Headings */
.heading {
    text-align: center;
    margin-bottom: 30px;
}

h2 {
    margin-top: 30px;
    margin-bottom: 10px;
    color: #2c3e50;
}

/* Action sections */
.actions-container {
    margin-bottom: 25px;
    padding: 15px;
    background-color: #fafafa;
    border: 1px solid #ddd;
    border-radius: 6px;
}

/* Inputs */
input[type="text"],
input[type="number"] {
    padding: 8px;
    margin: 5px 5px 5px 0;
    width: 200px;
    border: 1px solid #ccc;
    border-radius: 4px;
    font-size: 14px;
}

/* Buttons */
button {
    padding: 8px 14px;
    margin: 5px 5px 5px 0;
    background-color: #007bff;
    color: white;
    border: none;
    border-radius: 4px;
    font-size: 14px;
    cursor: pointer;
}

button:hover {
    background-color: #0056b3;
}
```

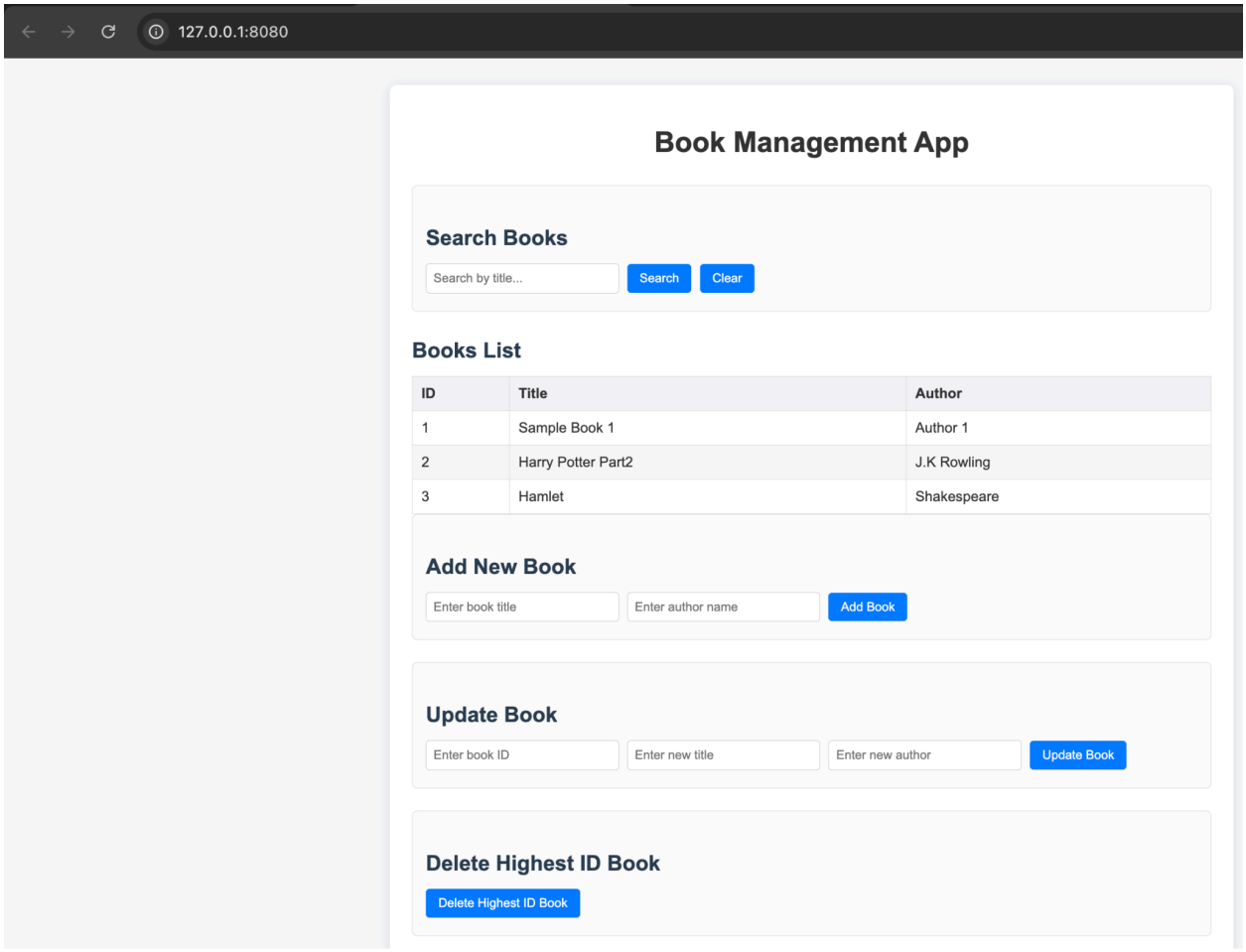
```css
/* Table styling */
.books-table {
    width: 100%;
    border-collapse: collapse;
    margin-top: 15px;
}

.books-table th,
.books-table td {
    padding: 10px;
    border: 1px solid #ddd;
    text-align: left;
}

.books-table th {
    background-color: #f0f2f5;
    font-weight: bold;
}

.books-table tr:nth-child(even) {
    background-color: #f9f9f9;
}

/* Responsive */
@media (max-width: 600px) {
    .container {
        width: 95%;
    }

    input[type="text"],
    input[type="number"] {
        width: 100%;
        margin-bottom: 10px;
    }

    button {
        width: 100%;
    }
}
```

Output: run “uvicorn main:app --reload --port 8080”





Add new book: Hamlet



2. Write the code to update the book with ID 1 to title:" Harry Potter", Author Name: "J.K Rowling". After submitting the data, redirect to the home view and show the updated data in the list of books.

   Explanation: Update functionality is implemented using PUT API /api/books/{book_id}. The backend first searches for the book with the given book id and returns an error if it doesn't exist. After the update is completed, loadbooks() is called to refresh the home view.

   Frontend: HTML

```
        <div class="actions-container">
            <h2>Update Book</h2>
                <input type="number" id="updateId" placeholder="Enter book ID"/>
                <input type="text" id="updateTitle" placeholder="Enter new
title"/>
                <input type="text" id="updateAuthor" placeholder="Enter new
author"/>
                <button onclick="updateBook()">Update Book</button>
        </div>
```

Javascript

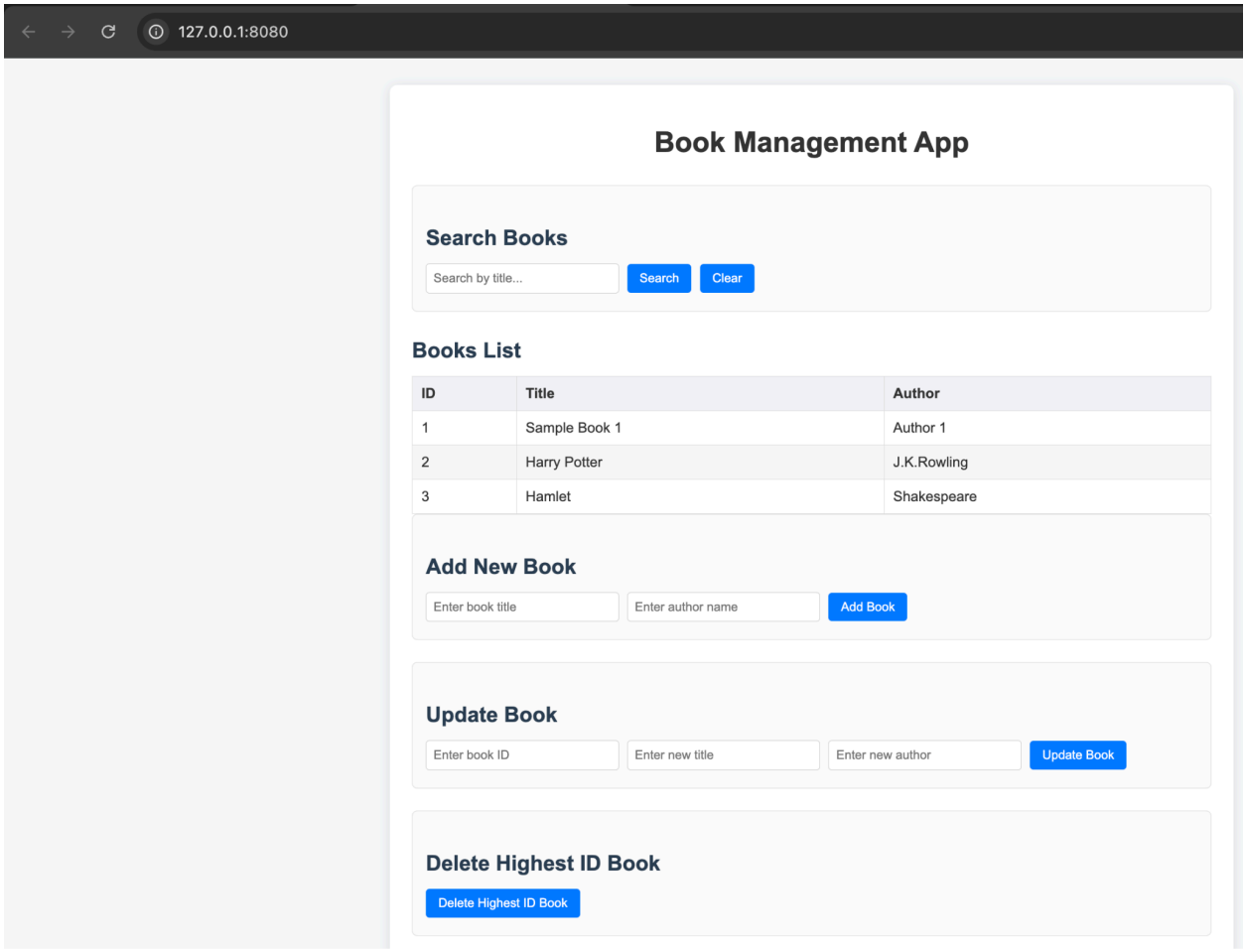
```javascript
async function updateBook() {
    const id = Number(document.getElementById('updateId').value);
    const title = document.getElementById('updateTitle').value.trim();
    const author = document.getElementById('updateAuthor').value.trim();
    if (!id || id <= 0) {
        alert('Please provide a valid book ID to update.');
        return;
    }
    if (!title && !author) {
        alert('Provide at least a new title or a new author to update.');
        return;
    }
    const body = {};
    if (title) body.title = title;
    if (author) body.author = author;
    try {
        const res = await fetch(`${api_base}/${id}`, {
            method: 'PUT',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify(body)
        });
        if (!res.ok) throw new Error('Failed to update book');
        document.getElementById('updateId').value = '';
        document.getElementById('updateTitle').value = '';
        document.getElementById('updateAuthor').value = '';
        await loadBooks();
    } catch (err) {
        console.error(err);
        alert('Could not update book. See console for details.');
    }
}
```

Backend (FastAPI)

```python
@app.put("/api/books/{book_id}", response_model=Book)
async def update_book(book_id: int, book_data: BookUpdate):
    book = next((b for b in books if b.id == book_id), None)
    if not book:
        raise HTTPException(status_code=404, detail="Book not found")

    if book_id == 1:
        book.title = "Harry Potter"
        book.author = "J.K Rowling"
        return book

    if book_data.title is None and book_data.author is None:
```

```
        raise HTTPException(status_code=400, detail="Provide title and/or author
to update")

    if book_data.title is not None:
        t = book_data.title.strip()
        if not t:
            raise HTTPException(status_code=400, detail="Title cannot be empty")
        book.title = t

    if book_data.author is not None:
        a = book_data.author.strip()
        if not a:
            raise HTTPException(status_code=400, detail="Author cannot be
empty")
        book.author = a

    return book
```

Output: Updated book id 2



3. Write the code to delete the book with the highest ID. After submitting the data, redirect to the home view and show the updated data in the list of books.

   Explanation: To delete the book with the highest ID, frontend first fetches all books from the backend and gets the highest id using reduce function and then Delete request is sent to /api/books/{book_id} with the highest id and removes it from the list.

   Frontend(HTML)

```
        <div class="actions-container">
            <h2>Delete Highest ID Book</h2>
                <button onclick="deleteHighestIdBook()">Delete Highest ID
Book</button>
        </div>
```

   Frontend (Javascript)

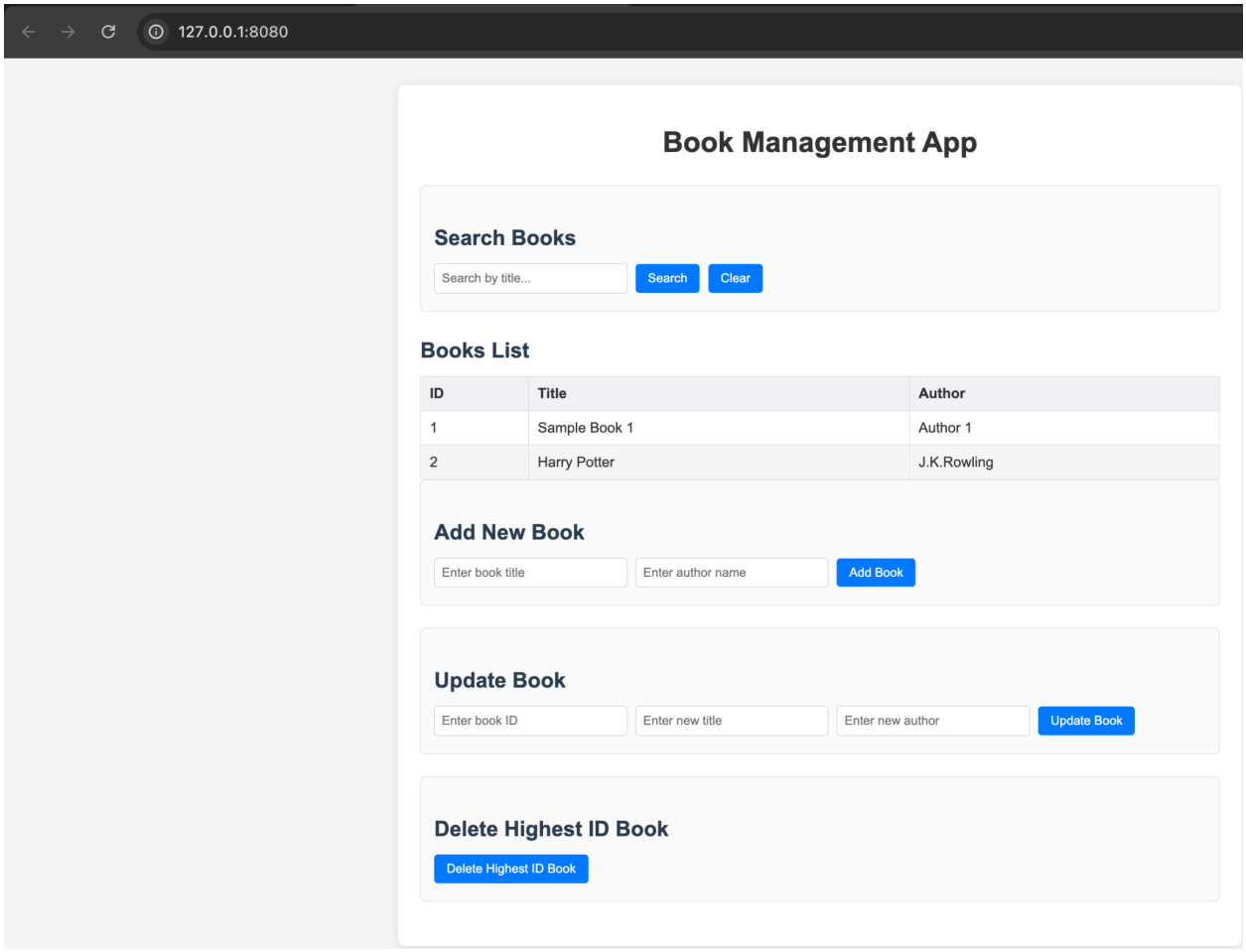
```
async function deleteHighestIdBook() {
    try {
        const res = await fetch(api_base);
        if (!res.ok) throw new Error('Failed to fetch books');
```

```
        const books = await res.json();
        if (!Array.isArray(books) || books.length === 0) {
            alert('No books available to delete.');
            return;
        }
        const max = books.reduce((acc, b) => {
            const idNum = Number(b.id);
            if (!isNaN(idNum) && idNum > acc) return idNum;
            return acc;
        }, -Infinity);
        if (!isFinite(max)) {
            alert('No valid numeric IDs found.');
            return;
        }
        const del = await fetch(`${api_base}/${max}`, { method: 'DELETE' });
        if (!del.ok) throw new Error('Failed to delete book');
        await loadBooks();
    } catch (err) {
        console.error(err);
        alert('Could not delete book. See console for details.');
    }
}
```

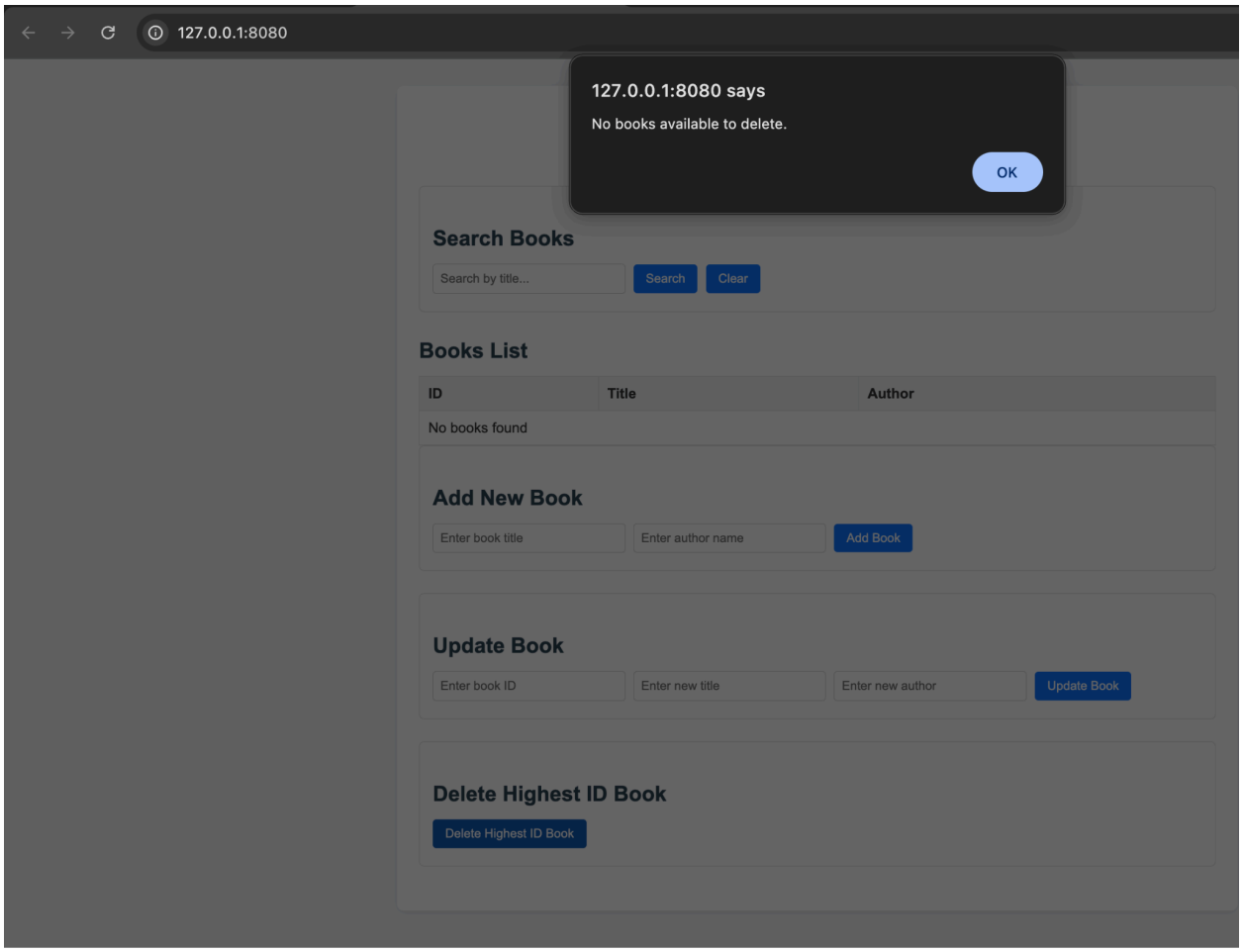
Backend (FastAPI)

```
@app.delete("/api/books/{book_id}", status_code=204)
async def delete_book(book_id: int):
    idx = next((i for i, b in enumerate(books) if b.id == book_id), None)
    if idx is None:
        raise HTTPException(status_code=404, detail="Book not found")
    books.pop(idx)
    return None
```

Output: Deleted highest ID book which was 3


127.0.0.1:8080
Book Management App
Search Books
Search by title...
Search
Clear
Books List
ID
Title
Author
1
Sample Book 1
Author 1
2
Harry Potter
J.K.Rowling
Add New Book
Enter book title
Enter author name
Add Book
Update Book
Enter book ID
Enter new title
Enter new author
Update Book
Delete Highest ID Book
Delete Highest ID Book

Output: If no books are available to delete, it will display “no books available to delete” message.



4. Write the code to add search functionality. The user should be able to search for a book by name/title. When the user enters a name and searches, the list should update to show only the matching books.

   Explanation: Search functionality is implemented on the frontend by reading the search text entered by the user and converting it to lowercase. All books are fetched from the backend and the list is filtered so that only books whose title contains the search text are kept. The function renderBooks() is then used to update the table and display only matching books.

   Frontend (HTML);

```
<div class="actions-container">
          <h2>Search Books</h2>
              <input type="text" id="searchTitle" placeholder="Search by
title..."/>
              <button onclick="searchBooks()">Search</button>
              <button onclick="loadBooks()">Clear</button>
      </div>
```

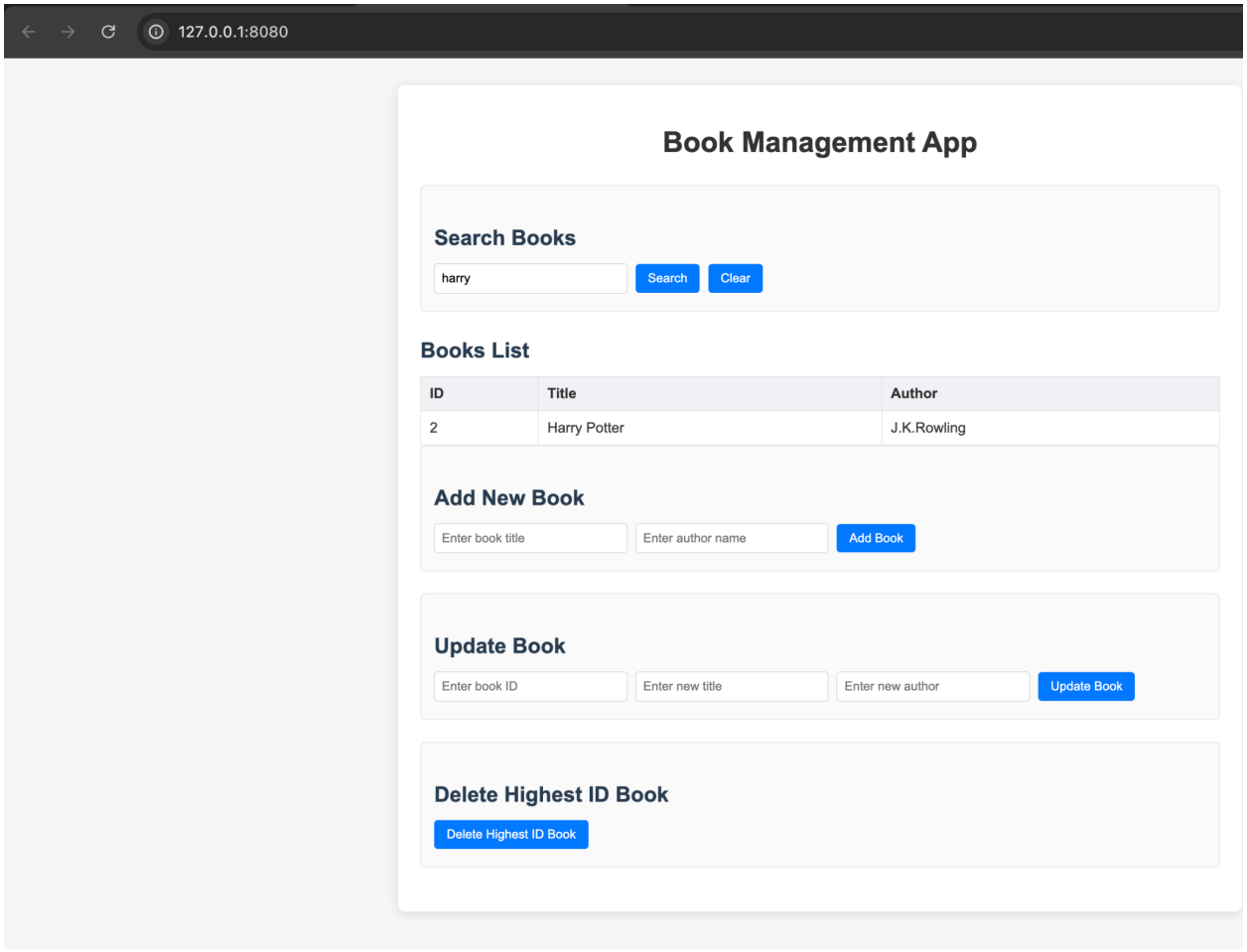
Frontend (Javascript)

```
async function searchBooks() {
   const q = document.getElementById('searchTitle').value.trim().toLowerCase();
   try {
       const res = await fetch(api_base);
       if (!res.ok) throw new Error('Failed to fetch books');
       const books = await res.json();
       if (!q) return renderBooks(books);
       const filtered = books.filter(b => (b.title || '').toLowerCase().includes(q));
       renderBooks(filtered);
   } catch (err) {
       console.error(err);
       alert('Search failed. See console for details.');
   }
}
```

Backend (FastAPI)

```
@app.get("/api/books", response_model=List[Book])
async def get_books(response: Response):
   response.headers["Cache-Control"] = "no-cache, no-store, must-revalidate"
   response.headers["Pragma"] = "no-cache"
   response.headers["Expires"] = "0"
   return books
```

Output: Search book title “harry”



Index.html:

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Book Management</title>
   <link rel="stylesheet" href="/static/css/style.css">
</head>
<body>
   <div class="container">
       <h1 class="heading">Book Management App</h1>
       <!--- Search Book Section -->
       <div class="actions-container">
           <h2>Search Books</h2>
               <input type="text" id="searchTitle" placeholder="Search by title..."/>
               <button onclick="searchBooks()">Search</button>
               <button onclick="loadBooks()">Clear</button>
       </div>
       <!--- Books List Section -->
       <h2>Books List</h2>
```

```
        <table id="booksTable" class="books-table">
            <thead>
                <tr>
                    <th>ID</th>
                    <th>Title</th>
                    <th>Author</th>
                </tr>
            </thead>
            <tbody id="booksTableBody">
            </tbody>
        </table>

        <!--- Add Book Section -->
        <div class="actions-container">
            <h2>Add New Book</h2>
                <input type="text" id="addTitle" placeholder="Enter book title"/>
                <input type="text" id="addAuthor" placeholder="Enter author name"/>
                <button onclick="addBook()">Add Book</button>
        </div>

        <!--- Update Book Section -->
        <div class="actions-container">
            <h2>Update Book</h2>
                <input type="number" id="updateId" placeholder="Enter book ID"/>
                <input type="text" id="updateTitle" placeholder="Enter new title"/>
                <input type="text" id="updateAuthor" placeholder="Enter new author"/>
                <button onclick="updateBook()">Update Book</button>
        </div>

        <!--- Delete book Section -->
        <div class="actions-container">
            <h2>Delete Highest ID Book</h2>
                <button onclick="deleteHighestIdBook()">Delete Highest ID Book</button>
        </div>
    </div>
    <script src="/static/js/app.js"></script>
</body>
</html>
```

main.py:

```
from fastapi import FastAPI, HTTPException, Response
from fastapi.staticfiles import StaticFiles
from fastapi.responses import FileResponse
from pydantic import BaseModel
from typing import List
import uvicorn
import webbrowser

app = FastAPI(title="Book Management API", version="1.0.0")

app.mount("/static", StaticFiles(directory="static"), name="static")
```

```python
# Models
class Book(BaseModel):
    id: int
    title: str
    author: str

class BookCreate(BaseModel):
    title: str
    author: str

class BookUpdate(BaseModel):
    title: str | None = None
    author: str | None = None

books: List[Book] = [
    Book(id=1, title="Sample Book 1", author="Author 1"),
    Book(id=2, title="Sample Book 2", author="Author 2"),
]

@app.get("/")
async def home():

    return FileResponse("templates/index.html")

@app.get("/api/books", response_model=List[Book])
async def get_books(response: Response):
    response.headers["Cache-Control"] = "no-cache, no-store, must-revalidate"
    response.headers["Pragma"] = "no-cache"
    response.headers["Expires"] = "0"
    return books

@app.post("/api/books", response_model=Book, status_code=201)
async def add_book(book_data: BookCreate):
    title = book_data.title.strip()
    author = book_data.author.strip()

    if not title:
        raise HTTPException(status_code=400, detail="Book title is required")
    if not author:
        raise HTTPException(status_code=400, detail="Author name is required")

    new_id = max([b.id for b in books], default=0) + 1
    new_book = Book(id=new_id, title=title, author=author)
    books.append(new_book)
    return new_book

@app.put("/api/books/{book_id}", response_model=Book)
async def update_book(book_id: int, book_data: BookUpdate):
    book = next((b for b in books if b.id == book_id), None)
    if not book:
        raise HTTPException(status_code=404, detail="Book not found")
```

```
    if book_id == 1:
        book.title = "Harry Potter"
        book.author = "J.K Rowling"
        return book

    if book_data.title is None and book_data.author is None:
        raise HTTPException(status_code=400, detail="Provide title and/or author to update")

    if book_data.title is not None:
        t = book_data.title.strip()
        if not t:
            raise HTTPException(status_code=400, detail="Title cannot be empty")
        book.title = t

    if book_data.author is not None:
        a = book_data.author.strip()
        if not a:
            raise HTTPException(status_code=400, detail="Author cannot be empty")
        book.author = a

    return book

@app.delete("/api/books/{book_id}", status_code=204)
async def delete_book(book_id: int):
    idx = next((i for i, b in enumerate(books) if b.id == book_id), None)
    if idx is None:
        raise HTTPException(status_code=404, detail="Book not found")
    books.pop(idx)
    return None

if __name__ == "__main__":
    webbrowser.open("http://localhost:8080")
    uvicorn.run(app, host="0.0.0.0", port=8080)
```

app.js

```
const api_base = '/api/books';


document.addEventListener('DOMContentLoaded', () => {
    loadBooks();
});


async function loadBooks() {
    try {
        const res = await fetch(api_base);
        if (!res.ok) throw new Error('Failed to fetch books');
        const books = await res.json();
        renderBooks(books);
    } catch (err) {
```

```
        console.error(err);
        alert('Could not load books. See console for details.');
    }
}

function renderBooks(books) {
    const tbody = document.getElementById('booksTableBody');
    tbody.innerHTML = '';
    if (!Array.isArray(books) || books.length === 0) {
        const row = document.createElement('tr');
        const cell = document.createElement('td');
        cell.colSpan = 3;
        cell.textContent = 'No books found';
        row.appendChild(cell);
        tbody.appendChild(row);
        return;
    }

    books.forEach(book => {
        const tr = document.createElement('tr');
        const idTd = document.createElement('td');
        idTd.textContent = book.id;
        const titleTd = document.createElement('td');
        titleTd.textContent = book.title;
        const authorTd = document.createElement('td');
        authorTd.textContent = book.author;

        tr.appendChild(idTd);
        tr.appendChild(titleTd);
        tr.appendChild(authorTd);
        tbody.appendChild(tr);
    });
}

// Search books by title
async function searchBooks() {
    const q = document.getElementById('searchTitle').value.trim().toLowerCase();
    try {
        const res = await fetch(api_base);
        if (!res.ok) throw new Error('Failed to fetch books');
        const books = await res.json();
        if (!q) return renderBooks(books);
```

```
        const filtered = books.filter(b => (b.title || '').toLowerCase().includes(q));
        renderBooks(filtered);
    } catch (err) {
        console.error(err);
        alert('Search failed. See console for details.');
    }
}

// Add new book
async function addBook() {
    const titleEl = document.getElementById('addTitle');
    const authorEl = document.getElementById('addAuthor');
    const title = titleEl.value.trim();
    const author = authorEl.value.trim();
    if (!title || !author) {
        alert('Please provide both title and author.');
        return;
    }
    try {
        const res = await fetch(api_base, {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ title, author })
        });
        if (!res.ok) throw new Error('Failed to add book');
        titleEl.value = '';
        authorEl.value = '';
        await loadBooks();
    } catch (err) {
        console.error(err);
        alert('Could not add book. See console for details.');
    }
}

// Update book by ID
async function updateBook() {
    const id = Number(document.getElementById('updateId').value);
    const title = document.getElementById('updateTitle').value.trim();
    const author = document.getElementById('updateAuthor').value.trim();
    if (!id || id <= 0) {
        alert('Please provide a valid book ID to update.');
        return;
```

```
    }
    if (!title && !author) {
        alert('Provide at least a new title or a new author to update.');
        return;
    }
    const body = {};
    if (title) body.title = title;
    if (author) body.author = author;
    try {
        const res = await fetch(`${api_base}/${id}`, {
            method: 'PUT',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify(body)
        });
        if (!res.ok) throw new Error('Failed to update book');
        document.getElementById('updateId').value = '';
        document.getElementById('updateTitle').value = '';
        document.getElementById('updateAuthor').value = '';
        await loadBooks();
    } catch (err) {
        console.error(err);
        alert('Could not update book. See console for details.');
    }
}

// Delete book with highest numeric ID
async function deleteHighestIdBook() {
    try {
        const res = await fetch(api_base);
        if (!res.ok) throw new Error('Failed to fetch books');
        const books = await res.json();
        if (!Array.isArray(books) || books.length === 0) {
            alert('No books available to delete.');
            return;
        }
        const max = books.reduce((acc, b) => {
            const idNum = Number(b.id);
            if (!isNaN(idNum) && idNum > acc) return idNum;
            return acc;
        }, -Infinity);
        if (!isFinite(max)) {
            alert('No valid numeric IDs found.');
```

```
            return;
        }
        const del = await fetch(`${api_base}/${max}`, { method: 'DELETE' });
        if (!del.ok) throw new Error('Failed to delete book');
        await loadBooks();
    } catch (err) {
        console.error(err);
        alert('Could not delete book. See console for details.');
    }
}
```

Part 3: Stateful Agent Graph

AgentState: This code defines a dictionary that represents the entire workflow state and provides a helper to initialize it before the agents start modifying it.

```
from __future__ import annotations

from typing import Any, Dict, TypedDict


class AgentState(TypedDict, total=False):
    title: str
    content: str
    email: str
    strict: bool
    task: str
    llm: Any
    planner_proposal: Dict[str, Any]
    reviewer_feedback: Dict[str, Any]
    reviewer_has_issues: bool
    turn_count: int
    interaction_log: list
    trace: list


def initialize_state(
    title: str,
    content: str,
    email: str,
    strict: bool,
    task: str,
    llm: Any = None,
) -> AgentState:
    return AgentState(
        title=title,
```

```
        content=content,
        email=email,
        strict=strict,
        task=task,
        llm=llm,
        planner_proposal={},
        reviewer_feedback={},
        turn_count=0,
        interaction_log=[],
        trace=[],
    )
```

Nodes:

Model: This function creates and returns a default local language model client using Ollama. It configures the model to use llama3.2:3b with temperature=0.5.

```
def _get_model_default() -> ChatOllama:
    return ChatOllama(
        model="llama3.2:3b",
        temperature=0.5,
    )
```

Planner Node: Planner node looks at the title and content and produces initial solution by generating tags and short summary.

```
def planner_node(state: AgentState) -> Dict[str, Any]:
    interaction_log = state.get("interaction_log", [])
    trace = state.get("trace", [])
    interaction_log.append("Planner: start")
    print("[Planner] Node activated")

    llm = _get_model_default()
    title = state.get("title", "")
    content = state.get("content", "")

    prompt = f"""You must respond with ONLY valid JSON. Nothing else.

    TEXT: {content}

        Create exactly:
        - 3 tags (1-3 words each, related to the content and title)
        - 1 summary (max 25 words)

        Your response must be valid JSON only:
        {{"tags": ["tag1", "tag2", "tag3"], "summary": "Summary here."}}"""

    try:
        t0 = time.time()
        msg = llm.invoke([HumanMessage(content=prompt)])
```

```
        t1 = time.time()
        content_response = getattr(msg, "content", "").strip()

        trace.append({
            "agent": "Planner",
            "type": "ai_message",
            "content": content_response,
            "metadata": {"latency_ms": int((t1 - t0) * 1000)},
            "ts": t1,
        })

        parsed = None

        try:
            parsed = json.loads(content_response)
        except Exception as err:
            print(f"[Planner] JSON parse error: {err}")

    except Exception as err:
        print(f"[Planner] ERROR: {err}")

    if parsed is None:
        parsed = {"tags": [], "summary": ""}

    interaction_log.append("Planner: complete")
    return {
        "planner_proposal": parsed,
        "reviewer_has_issues": None,
        "interaction_log": interaction_log,
        "trace": trace,
    }
```

Reviewer Node: is the checker. It takes what planner produced and evaluates it by checking if the result has exactly three tags and summary's word limit. Based on those checks, it decides whether the result is acceptable or not.

```
def reviewer_node(state: AgentState) -> Dict[str, Any]:
    interaction_log = state.get("interaction_log", [])
    trace = state.get("trace", [])
    interaction_log.append("Reviewer: start")
    print("[Reviewer] Node activated")

    llm = _get_model_default()
    title = state.get("title", "")
    content = state.get("content", "")
    proposal = state.get("planner_proposal", {})

    tags = proposal.get("tags", [])
    summary = proposal.get("summary", "")

    prompt = f"""You must respond with ONLY valid JSON. Nothing else.
```

```
    TEXT: {content}

    CURRENT TAGS: {tags}
    CURRENT SUMMARY: {summary}

    Validate and return:
    - 3 tags (each 1-3 words)
    - Summary (1-2 sentences, max 25 words)

    Response must be valid JSON only:
    {{"tags": ["tag1", "tag2", "tag3"], "summary": "Summary sentence."}}"""

    try:
        t0 = time.time()
        msg = llm.invoke([HumanMessage(content=prompt)])
        t1 = time.time()
        content_response = getattr(msg, "content", "").strip()
        trace.append({
            "agent": "Reviewer",
            "type": "ai_message",
            "content": content_response,
            "metadata": {"latency_ms": int((t1 - t0) * 1000)},
            "ts": t1,
        })

        try:
            result = json.loads(content_response)
        except:
            print(f"Reviewer Could not extract JSON, using current values")
            result = {"tags": tags, "summary": summary, "review_notes": ["Failed to fetch
review results"]}

    except Exception as err:
        print(f"[Reviewer] ERROR: {err}")
        result = {"tags": tags, "summary": summary, "review_notes": [str(err)]}

    interaction_log.append("Reviewer: complete")

    review_notes = []

    result_tags = result.get("tags", [])
    result_summary = result.get("summary", "")

    # Check tag count
    if len(result_tags) != 3:
        review_notes.append(f"Tags count is {len(result_tags)}, expected exactly 3")

    # Check summary word count
    summary_word_count = len(result_summary.split()) if result_summary else 0
    if summary_word_count > 25:
        review_notes.append(f"Summary has {summary_word_count} words, maximum is 25")
    if summary_word_count == 0:
```

```
        review_notes.append("Summary is empty")

    result["review_notes"] = review_notes

    # has issues if review_notes is not empty
    reviewer_has_issues = bool(review_notes)

    return {
        "reviewer_feedback": result,
        "reviewer_has_issues": reviewer_has_issues,
        "interaction_log": interaction_log,
        "trace": trace,
    }
```

Supervisor Node: is the controller. Combined with the router, it determines whether the system should ask the planner to try again, send the result to the reviewer or stop the workflow entirely. Supervisor will increment the turn count.

```
def supervisor_node(state: AgentState) -> Dict[str, Any]:
    """Increments turn counter.

    Returns updated `turn_count`.
    """
    turn = state.get("turn_count", 0) + 1
    return {"turn_count": turn}
```

Router.py: is the decision maker for the control flow. It looks at the current state and decides whether to run the planner, run the reviewer, or end the workflow based on whether a proposal exists, whether the reviewer found issues, and whether the turn limit has been reached.

```
from __future__ import annotations

from typing import Literal

from state import AgentState

def router_logic(state: AgentState) -> Literal["run_planning", "run_review", "END"]:
    turn = state.get("turn_count", 0)
    has_proposal = bool(state.get("planner_proposal"))
    has_reviewer_issues_key = "reviewer_has_issues" in state
    reviewer_has_issues = state.get("reviewer_has_issues", None) if has_reviewer_issues_key
else None

    print(f"[Router] turn={turn}, has_proposal={has_proposal},
reviewer_has_issues={reviewer_has_issues}")

    if turn > 6:
        print(f"[Router] -> END (turn > 6)")
        return "END"

    if not has_proposal:
        print(f"[Router] -> run_planning (no proposal)")
```

```
        return "run_planning"

    # If reviewer_has_issues is True, go back to planning
    if reviewer_has_issues is True:
        print(f"[Router] -> run_planning (reviewer has issues)")
        return "run_planning"

    # If reviewer_has_issues is False, end workflow
    if reviewer_has_issues is False:
        print(f"[Router] -> END (reviewer has no issues)")
        return "END"

    # For None, run the review again.
    print(f"[Router] -> run_review (has proposal, review needed)")
    return "run_review"
```

Workflow.py: It defines how the nodes are connected, in what order they can run and how control flows between them. Execution always starts at the supervisor, which uses the router logic to decide whether to run the planner, run the reviewer, or end the workflow. After either the planner or reviewer finishes, control always returns to the supervisor until the workflow ends.

```
from __future__ import annotations

from langgraph.graph import StateGraph, END

from state import AgentState
from nodes import planner_node, reviewer_node, supervisor_node
from router import router_logic


def build_workflow():
    workflow = StateGraph(AgentState)

    workflow.add_node("supervisor", supervisor_node)
    workflow.add_node("planner", planner_node)
    workflow.add_node("reviewer", reviewer_node)

    workflow.set_entry_point("supervisor")

    # Supervisor decides what to do next
    workflow.add_conditional_edges(
        "supervisor",
        router_logic,
        {
            "run_planning": "planner",
            "run_review": "reviewer",
            "END": END,
        },
    )

    # Planner always goes to supervisor after completion
```

```
    workflow.add_edge("planner", "supervisor")

    # Reviewer always goes to supervisor after completion
    workflow.add_edge("reviewer", "supervisor")

    return workflow.compile()
```

Test.py: Test_workflow_stream() function builds the workflow, initializes the starting state with sample content, and then runs the workflow step by step using streaming. As each node executes, it prints the current turn, which node ran, and any planner or reviewer outputs so you can see how the state evolves over time. In the end, it shows that the workflow has completed or prints an error if something goes wrong.

```
from state import initialize_state
from workflow import build_workflow


def test_workflow_stream():

    app = build_workflow()

    title = "Build and run docker Image"
    content = """Docker Image is a read-only template to build containers. An image holds all
    the information needed to bootstrap a container, including what processes
    to run and the configuration data. Every image starts from a base image,
    and a template is created by using the instructions that are stored in the
    DockerFile. For each instruction, a new layer is created on the image.
    Running the container originates from the image we created in the previous
    step. When a container is launched, a read-write layer is added to the top of the
    image. After appropriate network and IP address allocation, the desired
    application can now be run inside the container."""

    initial_state = initialize_state(
        title=title,
        content=content,
        email="test@example.com",
        strict=True,
        task="Extract tags and summary from content"
    )

    print("\n" + "=" * 80)
    print("WORKFLOW EXECUTION")
    print("=" * 80)
    print(f"Title: {title}\n")

    try:
        current_turn = 0

        for output in app.stream(initial_state):
            node_name = list(output.keys())[0]
```

```
            node_state = output[node_name]

            # Get turn count from state (supervisor always has it)
            if node_name == "supervisor":
                current_turn = node_state.get('turn_count', 0)

            has_issues = node_state.get('reviewer_has_issues', None)
            proposal = node_state.get("planner_proposal", {})
            feedback = node_state.get("reviewer_feedback", {})

            print(f"\n--- Turn {current_turn}: {node_name.upper()} ---")

            if node_name == "supervisor":
                print(f"  (Turn count: {current_turn})")

            elif node_name == "planner":
                tags = proposal.get('tags', [])
                summary = proposal.get('summary', '')
                print(f"  Tags: {tags}")
                print(f"  Summary: {summary}")

            elif node_name == "reviewer":
                tags = feedback.get('tags', [])
                summary = feedback.get('summary', '')
                notes = feedback.get('review_notes', [])
                print(f"  Tags: {tags}")
                print(f"  Summary: {summary}")
                print(f"  Has issues: {has_issues}")
                if notes:
                    print(f"  Review notes: {notes}")

        print(f"\n" + "=" * 80)
        print(f"WORKFLOW COMPLETED")
        print(f"=" * 80)

    except Exception as e:
        print(f"\nERROR: {e}")
        import traceback
        traceback.print_exc()


if __name__ == "__main__":
    test_workflow_stream()
```

<u>Output 1:</u> Single successful run of the workflow

1. Workflow starts and the router checks the state. Since there is no planner proposal yet, it decides to run the planner.

2. Turn 1- supervisor increments the turn count, then the planner node runs. The planner generates 3 tags and a short summary. After that router runs again and sees that proposal now exists but has not been reviewed yet, so it sends the workflow to the reviewer
3. Turn 2- supervisor increments the turn count again, then reviewer node runs. The reviewer checks the planner's tag and summary and finds no issues i.e reviewer_has_issues=False
4. The router runs again, sees the reviewer has no issues and decides to end the workflow.

```
(env) Kruthika@Kruthikas-MacBook-Air Part2 % python test.py

================================================================================
WORKFLOW EXECUTION
================================================================================
Title: Build and run docker Image

[Router] turn=1, has_proposal=False, reviewer_has_issues=None
[Router] -> run_planning (no proposal)

--- Turn 1: SUPERVISOR ---
  (Turn count: 1)
[Planner] Node activated

--- Turn 1: PLANNER ---
  Tags: ['Docker Image', 'Container Build', 'Template Creation']
  Summary: A read-only template to build containers with all necessary information and configuration data.
[Router] turn=2, has_proposal=True, reviewer_has_issues=None
[Router] -> run_review (has proposal, review needed)

--- Turn 2: SUPERVISOR ---
  (Turn count: 2)
[Reviewer] Node activated

--- Turn 2: REVIEWER ---
  Tags: ['Docker Image', 'Container Build', 'Template Creation']
  Summary: A read-only template to build containers with all necessary information and configuration data.
  Has issues: False
[Router] turn=3, has_proposal=True, reviewer_has_issues=False
[Router] -> END (reviewer has no issues)

--- Turn 3: SUPERVISOR ---
  (Turn count: 3)

================================================================================
WORKFLOW COMPLETED
================================================================================
(env) Kruthika@Kruthikas-MacBook-Air Part2 %
```

<u>Output 2</u>: Correction loop output

In nodes.py, I intentionally force the correction loop by setting reviewer_has_issue=True inside reviewer node. Now the reviewer never approves a proposal and the planner /reviewer loops until the turn limit end the workflow.

```python
def reviewer_node(state: AgentState) -> Dict[str, Any]:
    interaction_log = state.get("interaction_log", [])
    trace = state.get("trace", [])
    interaction_log.append("Reviewer: start")
    print("[Reviewer] Node activated")

    llm = _get_model_default()
    title = state.get("title", "")
    content = state.get("content", "")
    proposal = state.get("planner_proposal", {})
```

```
    tags = proposal.get("tags", [])
    summary = proposal.get("summary", "")

    prompt = f"""You must respond with ONLY valid JSON. Nothing else.

    TEXT: {content}

    CURRENT TAGS: {tags}
    CURRENT SUMMARY: {summary}

    Validate and return:
    - 3 tags (each 1-3 words)
    - Summary (1-2 sentences, max 25 words)

    Response must be valid JSON only:
    {{"tags": ["tag1", "tag2", "tag3"], "summary": "Summary sentence."}}"""

    try:
        t0 = time.time()
        msg = llm.invoke([HumanMessage(content=prompt)])
        t1 = time.time()
        content_response = getattr(msg, "content", "").strip()
        trace.append({
            "agent": "Reviewer",
            "type": "ai_message",
            "content": content_response,
            "metadata": {"latency_ms": int((t1 - t0) * 1000)},
            "ts": t1,
        })

        try:
            result = json.loads(content_response)
        except:
            print(f"Reviewer Could not extract JSON, using current values")
            result = {"tags": tags, "summary": summary, "review_notes": ["Failed to fetch
review results"]}

    except Exception as err:
        print(f"[Reviewer] ERROR: {err}")
        result = {"tags": tags, "summary": summary, "review_notes": [str(err)]}

    interaction_log.append("Reviewer: complete")

    review_notes = []

    result_tags = result.get("tags", [])
    result_summary = result.get("summary", "")

    # Check tag count
    if len(result_tags) != 3:
        review_notes.append(f"Tags count is {len(result_tags)}, expected exactly 3")
```

```
    # Check summary word count
    summary_word_count = len(result_summary.split()) if result_summary else 0
    if summary_word_count > 25:
        review_notes.append(f"Summary has {summary_word_count} words, maximum is 25")
    if summary_word_count == 0:
        review_notes.append("Summary is empty")

    result["review_notes"] = review_notes

    # Force set to true to check the feedback loop
    reviewer_has_issues = True

    return {
        "reviewer_feedback": result,
        "reviewer_has_issues": reviewer_has_issues,
        "interaction_log": interaction_log,
        "trace": trace,
    }
```

This output shows the planner and reviewer repeatedly looping because the reviewer is forced to report issues, until the supervisor and router stop the workflow after reaching the maximum turn limit.

```
(env) Kruthika@Kruthikas-MacBook-Air Part2 % python test.py

================================================================================
WORKFLOW EXECUTION
================================================================================
Title: Build and run docker Image

[Router] turn=1, has_proposal=False, reviewer_has_issues=None
[Router] -> run_planning (no proposal)

--- Turn 1: SUPERVISOR ---
  (Turn count: 1)
[Planner] Node activated

--- Turn 1: PLANNER ---
  Tags: ['docker image', 'container bootstrapping', 'image creation']
  Summary: Docker Image: A read-only template for building containers with configuration data and process instructions.
[Router] turn=2, has_proposal=True, reviewer_has_issues=None
[Router] -> run_review (has proposal, review needed)

--- Turn 2: SUPERVISOR ---
  (Turn count: 2)
[Reviewer] Node activated

--- Turn 2: REVIEWER ---
  Tags: ['docker image', 'container bootstrapping', 'image creation']
  Summary: A read-only template for building containers with configuration data and process instructions.
  Has issues: True
  Review notes: ['Just a force issue to log an issue in Reviewer!']
[Router] turn=3, has_proposal=True, reviewer_has_issues=True
[Router] -> run_planning (reviewer has issues)

--- Turn 3: SUPERVISOR ---
  (Turn count: 3)
[Planner] Node activated

--- Turn 3: PLANNER ---
  Tags: ['docker image', 'container bootstrap', 'template creation']
  Summary: A read-only template to build and run containers, including processes and configuration data.
[Router] turn=4, has_proposal=True, reviewer_has_issues=None
[Router] -> run_review (has proposal, review needed)

--- Turn 4: SUPERVISOR ---
  (Turn count: 4)
[Reviewer] Node activated

--- Turn 4: REVIEWER ---
  Tags: ['docker image', 'container bootstrap', 'template creation']
  Summary: A read-only template to build and run containers.
  Has issues: True
  Review notes: ['Just a force issue to log an issue in Reviewer!']
[Router] turn=5, has_proposal=True, reviewer_has_issues=True
[Router] -> run_planning (reviewer has issues)

--- Turn 5: SUPERVISOR ---
  (Turn count: 5)
[Planner] Node activated

--- Turn 5: PLANNER ---
  Tags: ['docker image', 'container', 'building']
  Summary: A template to build containers with all necessary information.
[Router] turn=6, has_proposal=True, reviewer_has_issues=None
[Router] -> run_review (has proposal, review needed)

--- Turn 6: SUPERVISOR ---
  (Turn count: 6)
[Reviewer] Node activated

--- Turn 6: REVIEWER ---
  Tags: ['Docker Image', 'Container', 'Building']
  Summary: A template to build containers with all necessary information.
  Has issues: True
  Review notes: ['Just a force issue to log an issue in Reviewer!']
[Router] turn=7, has_proposal=True, reviewer_has_issues=True
[Router] -> END (turn > 6)

--- Turn 7: SUPERVISOR ---
  (Turn count: 7)

================================================================================
WORKFLOW COMPLETED
================================================================================
(env) Kruthika@Kruthikas-MacBook-Air Part2 %
```